

Solutions to FIT3080 Applied Class on Problem Solving as Search: Graphsearch – Heuristic Search

Exercise 1: Algorithm A/A*, Admissibility

- (a) Is Algorithm A the same Algorithm as A*?

SOLUTION:

A is not the same as A*, as

- A requires $h(n) \geq 0 \forall n$, while
- A* requires $0 \leq h(n) \leq h^*(n) \forall n$.

- (b) Show that if $h(n)$ overestimates $h^*(n)$, Algorithm A may return a non-optimal solution. You can use an example.

SOLUTION:

Say we have two nodes A and B . Assume

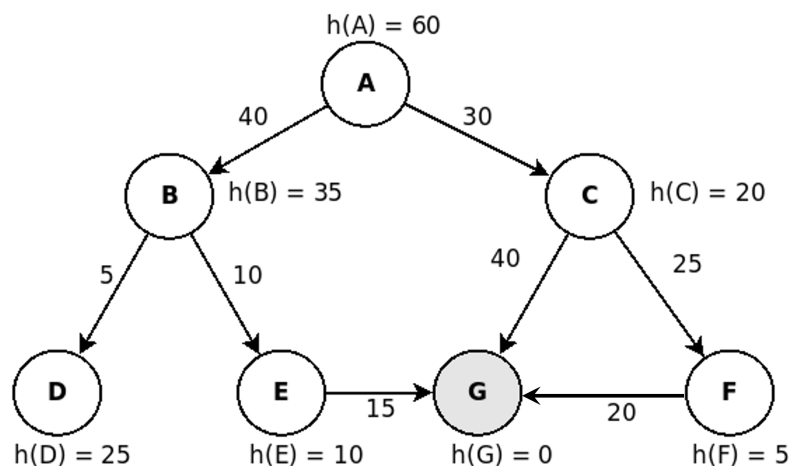
- $g^*(A) = g(A) = g(B) = g^*(B) = g$,
- $h^*(A) = 8$ and $h^*(B) = 10$, but
- $h(A) = 12 > h^*(A)$ and $h(B) = 11 > h^*(B)$.

So, $f(A) = g(A) + h(A) = g + 12$ and $f(B) = g(B) + h(B) = g + 11$. Therefore, we expand B , reaching the goal with $f^*(B) = g + 10$.

Since $f^*(B) = g + 10 < f(A) = g(A) + h(A) = g + 12$, A is not expanded, and the optimal solution of $g^*(A) + h^*(A) = g + 8$ is not found.

Exercise 2: Informed and Uninformed Search Strategies

Consider the (full) state space below, where the goal node is **G**. For each of the following search strategies, show the FRONTIER in the order nodes are examined (IDDFS) or in order of merit (Greedy BFS, A and A*), which nodes remain in memory after the search ends, and what is the solution and its cost. For IDDFS also indicate the order in which nodes are created, and for Greedy BFS, Algorithm A and A*, also show EXPLORED.



- (a) Iterative deepening depth-first search

SOLUTION:

Assuming a left-right-right traversal of the successors, IDDFS expands:

Iteration	Limit	FRONTIER	Created
1	0	{ A }	
2	1	{ A } { B, C }	{ B, C }
3	2	{ A } { B, C } { D, E, C } { E, C } { C } { G, F }	{ B, C } { D, E }

In iteration 2, B is deleted, then C and then A.

In iteration 3, D is deleted, then E, then B, then C is expanded, and the goal is found.

The nodes in memory are: A, C, **G** and F.

The cost of the solution A-C-**G** is 70.

- (b) Greedy best-first search

SOLUTION:

GBFS expands:

Iteration	FRONTIER	EXPLORED
0	{ (A, 60) }	
1	{ (C, 20), (B, 35) }	{ (A, 60) }
2	{ (G , 0), (F, 5), (B, 35) }	{ (A, 60), (C, 20) }
3	{ (F, 5), (B, 35) }	{ (A, 60), (C, 20), (G , 0) }

The nodes in memory are: A, B, C, **G** and F.

The cost of the solution A-C-**G** is 70.

- (c) Algorithm A (as Graphsearch or Treesearch)

SOLUTION:

Algorithm A expands:

Iteration	FRONTIER	EXPLORED
0	{ (A, 60) }	
1	{ (C, 30+20=50), (B, 40+35=75) }	{ (A, 60) }
2	{ (F, 55+5=60), (G , 70+0=70), (B, 75) }	{ (A, 60), (C, 50) }
3	{ (G , 70), (B, 75) }	{ (A, 60), (C, 50), (F, 60) }
4	{ (B, 75) }	{ (A, 60), (C, 50), (F, 60), (G , 70) }

After node F is expanded, **G** is reached with a cost of 75, so in Graphsearch, the lower cost of 70 is retained. In Treesearch, there are two **G**s, one with cost 70 and one with cost 75. The one with cost 70 is considered first.

The nodes in memory are: A, B, C, **G** and F.

The cost of the solution A-C-**G** is 70.

- (d) What modifications are needed to turn Algorithm A into A*?

SOLUTION:

We need to modify the heuristic to be admissible. The node where the heuristic over-estimates the true cost is B. We have $h(B) = 35$ but $h^*(B) = 25$. Any value of $h(B)$ between 0 and 25 would work.

- (e) After the modification, which nodes are expanded by A*?

SOLUTION:

Suppose $h(B)$ now returns 25. A* expands:

It.	FRONTIER	EXPLORED
0	{ (A, 60) }	
1	{ (C, 30+20=50), (B, 40+25=65) }	{ (A, 60) }
2	{ (F, 55+5=60), (B, 65), (G , 70+0=70) }	{ (A, 60), (C, 50) }
3	{ (B, 65), (G , 70) }	{ (A, 60), (C, 50), (F, 60) }
4	{ (E, 50+10=60), (G , 70), (D, 45+25=70) }	{ (A, 60), (C, 50), (F, 60), (B, 65) }
5	{ (G , 65), (D, 45+25=70) }	{ (A, 60), (C, 50), (F, 60), (B, 65), (E, 60) }

After node F is expanded, **G** is reached with a cost of 75, so in Graphsearch, the lower cost of 70 is retained.

The tie break between **G** and D is arbitrary.

After node E is expanded, **G** is reached with a cost of 65, so in Graphsearch, the lower cost of 65 is retained.

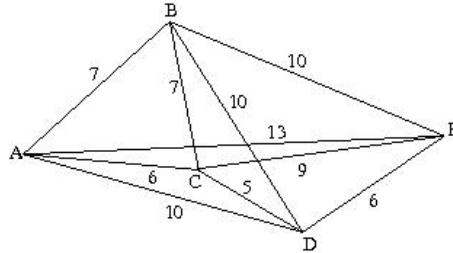
The nodes in memory are: A, B, C, D, E, **G** and F.

The cost of the solution A-B-E-**G** is 65.

Exercise 3: Informed Search Algorithms – Greedy-best-first-search/A/A*

Consider the Traveling Salesman Problem:

A salesman must visit each of n cities. There is a road between each pair of cities. Starting at city A, find the route of minimal distance that visits each of the cities only once and returns to city A.



- (a) Propose two (non-zero) h functions for this problem. Is either of these h functions a lower bound of h^* ?

SOLUTION:

Algorithm A is a type of Graphsearch algorithm which uses the following function to determine which node to expand next:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ - is the **cumulative** cost of arriving from the start node to node n , i.e., the cost of the entire path from start to n .
- $h(n)$ - is an estimate of the cost of arriving from node n to the goal node. This estimate does **not** include the cost of arriving to node n .

If we can infer that this estimate is always less than the cost of the best path from node n to the goal (which we don't know at the moment), then A becomes A*.

Thus, $f(n)$ is a composite measure of the cost incurred in reaching node n plus the cost we are expecting to incur in order to arrive from node n to the goal. In other words, it is a measure of the cost of getting from the start to the goal **through node n**.

Keep in mind that algorithm A **must consider the cost of arriving at node n**. If it disregards this cost, then it is merely considering the promise of a particular node (e.g., node n), without taking into consideration how much it costs to arrive at this node. This would yield *Greedy-best-first-search*, and by definition, it would no longer be algorithm A.

HEURISTIC FUNCTION

There are several heuristic functions that can be applied to this problem, we shall now present a few, and thereafter we shall present the search-tree reached for the last of these functions.

1. h_1 = Number of cities left to be visited.

This function represents a lower bound to h^* . However it does not provide information with respect to the relative promise of each path, since at each level the remaining number of cities to be visited is the same.

2. h_2 = Distance to the closest city.

This function is also less than h^* , since it takes into consideration only the closest city (which, clearly, has to be visited). It disregards the constraint that all cities should be visited.

- 3.

$$h_3 = \sum_{i \in C'} \min_{x \in \text{neighbour}(i)} (\text{dist}(x \rightarrow i)) \quad (1)$$

where $C' = \{\text{cities left to be visited}\}$.

This function takes into consideration the minimum distance to each city in C' . It disregards the constraint that we have to leave the city where we are now and the fact that we cannot depart from most previously visited cities, except the last one. It is also a lower bound of h^* .

- 4.

$$h_4 = \sum_{i \in C'} \min_{x \in C''} (\text{dist}(x \rightarrow i)) \quad (2)$$

where $C' = \text{cities not yet visited}$, and

- For $i \neq A$: $C'' = \{C' - A\} \cup \{\text{last-city-visited}\}$

For each city i not yet visited, except A , which is visited at the end, we consider the distance from $\{\text{all the not-yet-visited neighbours of } i \text{ plus the last visited city}\}$ to i .

- For $i = A$ and $|C'| > 1$: $C'' = \{C' - A\}$
- For $i = A$ and $|C'| = 1$: $C'' = \{\text{last city visited}\}$

The difference between h_3 and h_4 is that h_3 considers the minimum distance to a particular city **from all the cities**, while h_4 takes into consideration only the cities that remain to be visited + the last city visited (except for A , which must be the last city to be visited). Thus, the estimate provided by h_4 is more accurate than the one provided by h_3 , but still is a lower bound of h^* .

For example, in the given problem, if we already visited ABC , in this order, and D,E and A remain to be visited.

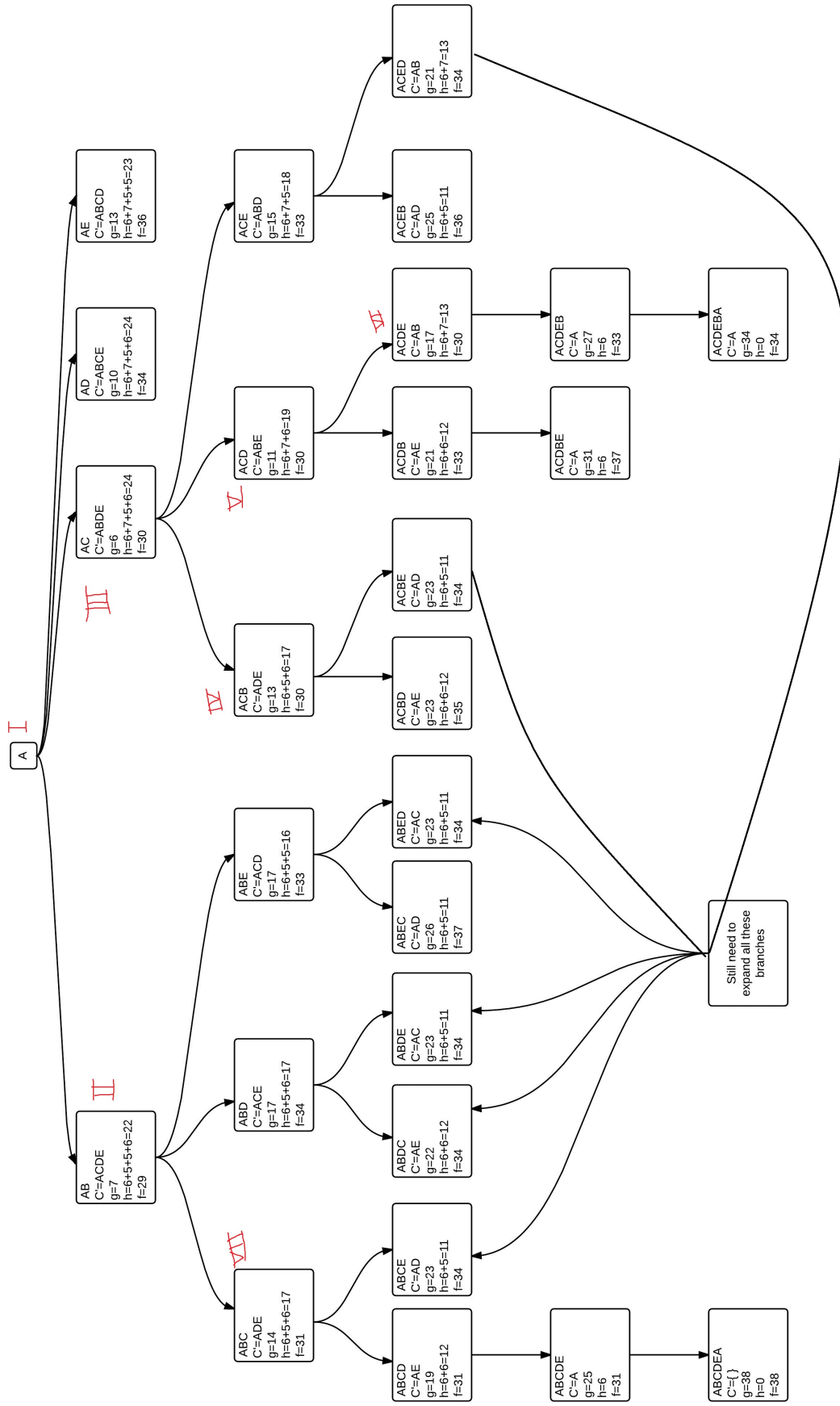
$$\begin{aligned} h_3 &= \min(AD, BD, CD, ED) + \min(AE, BE, CE, DE) + \\ &\quad \min(BA, CA, DA, EA) \\ &= 5 + 6 + 6 = 17 \end{aligned}$$

$$h_4 = \min(CD, ED) + \min(CE, DE) + \min(DA, EA) = 5 + 6 + 10 = 21$$

- (b) Apply algorithm A with these h functions to the 5 city problem above.

SOLUTION:

The following diagram contains the search tree produced by algorithm A when using h_3 .



The following diagram contains the search tree produced by algorithm A when using h_4 .

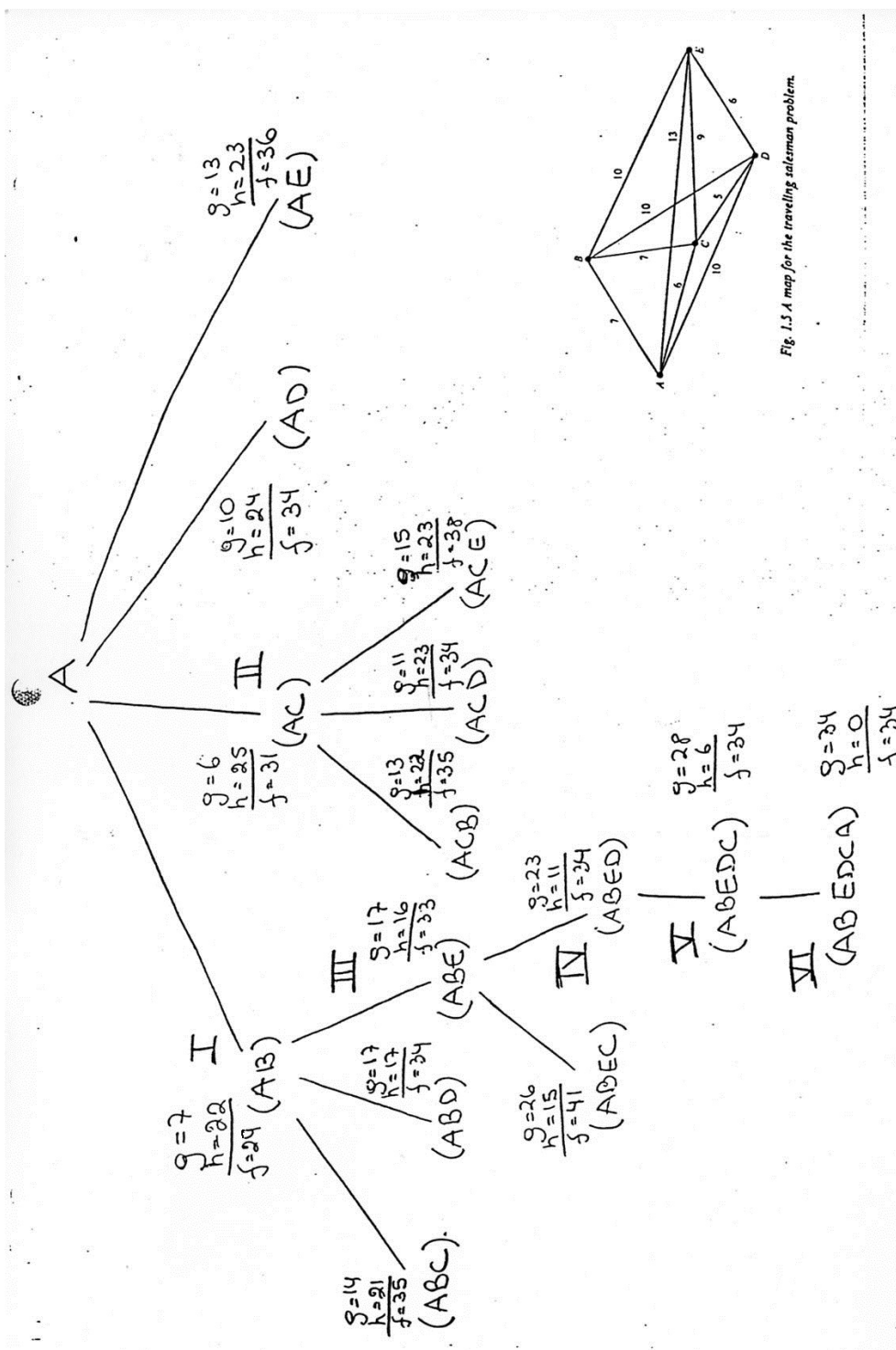


Fig. 1.3 A map for the traveling salesman problem.

- (c) How would your solution change if you used Greedy best first search instead of algorithm A?

SOLUTION:

Looking at the searchtree for h_3 ,

1. Children of A are $\{AB, AC, AE\}$. AB would be expanded first ($h_3 = 22$).
2. Children of AB are $\{ABC, ABD, ABE\}$. ABE would be expanded second ($h_3 = 16$).
3. Children of ABE are $\{ABEC, ABED\}$. Both have the same $h_3 = 11$.

Exercise 4: Algorithm A*

Consider a modified version of the 8-puzzle, where moving the blank tile to the center square has a cost of 4 (the rest of the moves cost 1). The initial configuration is:

2	8	3
1	6	4
7		5

Where the goal 8-puzzle is the following:

1	2	3
8		4
7	6	5

- (a) Apply algorithm A* to find the shortest path between the initial and the final configuration. Use Manhattan Distance as your heuristic function.

In the Search Tree produced by the algorithm, indicate clearly the order of expansion of each node (i.e., when is a node **expanded**, not generated), and the values of g , h and f . Label the moves clearly.

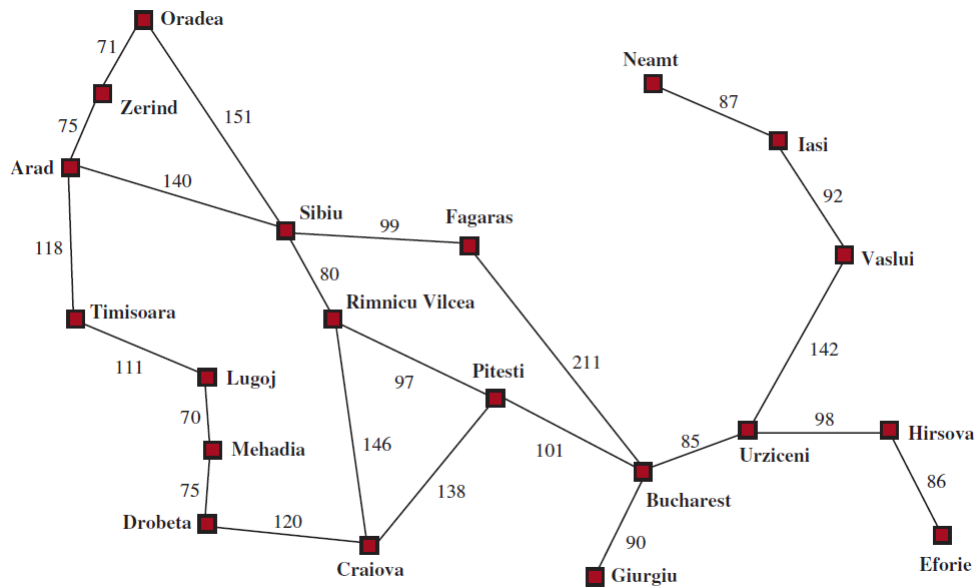
- (b) Describe what happens upon reaching the goal state. If you think that the heuristic function of A* requires any modifications to improve its efficiency for this problem, describe and justify the modifications you envisage.

SOLUTION:

- (a) The solution tree is shown below.

Exercise 5: Algorithm A* (Optional, Python Coding)

Once again, consider the Romania problem of finding a path from Arad to Bucharest.



Using the edge costs and the heuristic function from `lab3_astar_template.py` on Moodle, implement the A* algorithm to find a path from Arad to Bucharest.

SOLUTION:

See the code file `lab3_astar_solution.py` on Moodle.